

Project Setup (IntelliJ/Gradle)

This page provides instructions for setting up IntelliJ/Gradle for use in this class and how to run code/tests.

Important: We will build and run submissions using Java 25, without preview features (as these are not guaranteed to be part of future Java versions). Your language level should be set to `25 - Compact source files, module imports` in Project Settings.

Important: Gradle is notorious for weird environment issues. You should setup an environment ASAP to allow time to debug potential issues in office hours if required. Environment issues are NOT an excuse for being unable to complete assignments!

IntelliJ IDEA

IntelliJ is the recommended IDE because it natively supports Java, Gradle, and JUnit (which will all be used in this class). This is the easiest method to run both your project and the provided tests and is what will be used in lecture.

- **IntelliJ IDEA Download** (<https://www.jetbrains.com/idea/download>)
 - Either the Ultimate or Community edition is fine. As students, you can apply for a free license to all JetBrains products which may be helpful outside this class as well ([Student License](https://www.jetbrains.com/community/education)). (<https://www.jetbrains.com/community/education>)

While we're not going to strictly require IntelliJ, you will need to have environments set up to work with the above tools. At the very least, we recommend using IntelliJ for setting up the project and testing your submission and then whatever else you'd like for editing code.

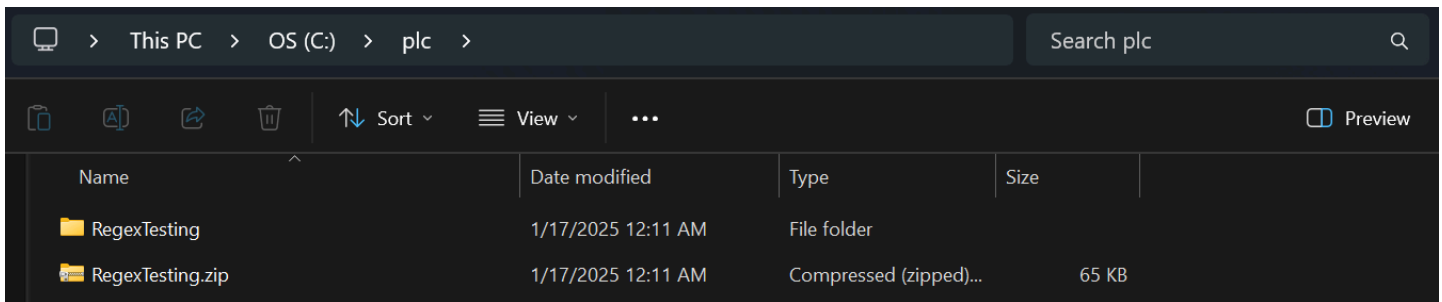
TA's will be prepared to assist with IntelliJ, *not* custom [Haiku installs \(https://xkcd.com/806/\)](https://xkcd.com/806/).
YMMV

Project Setup

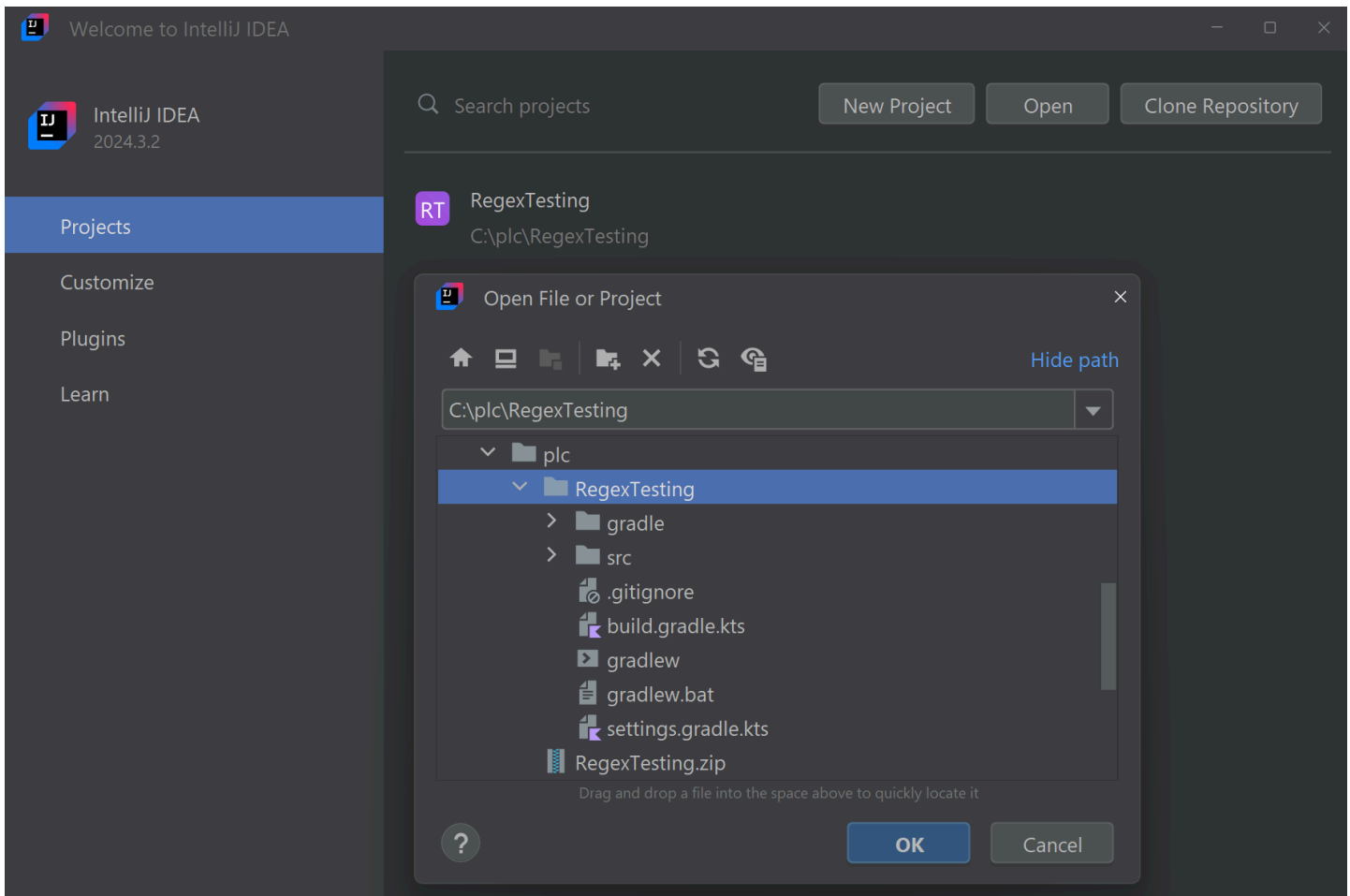
Each assignment will have a separate zip with provided source files. The example below uses [RegexTesting.zip \(https://ufl.instructure.com/courses/555493/files/103740458?wrap=1\)](https://ufl.instructure.com/courses/555493/files/103740458?wrap=1) ↓ (https://ufl.instructure.com/courses/555493/files/103740458/download?download_frd=1) from [Exercise 1: Regex Testing \(https://ufl.instructure.com/courses/555493/assignments/6905742\)](https://ufl.instructure.com/courses/555493/assignments/6905742). The screenshots below use IntelliJ Community Edition with the classic UI, so if you're using the newer style you may need to search a bit for the settings.

These projects are setup using [Gradle \(https://gradle.org\)](https://gradle.org), which is one of the main build systems for Java (most commonly known for Android development). Gradle is used to manage project dependencies for us, such as JUnit for testing. Gradle can be quite tricky and is notorious for weird environment issues, but with any luck it should "just work". If you experience issues, ask in office hours (and start early to allow time!).

To start, download and unzip the project files to your desired directory (in the example, `C:\plc`). You should **avoid** directories that are cloud syncs (e.g. OneDrive); these are known to cause issues and generally not recommend for most development (use a **private** GitHub repository for backups).

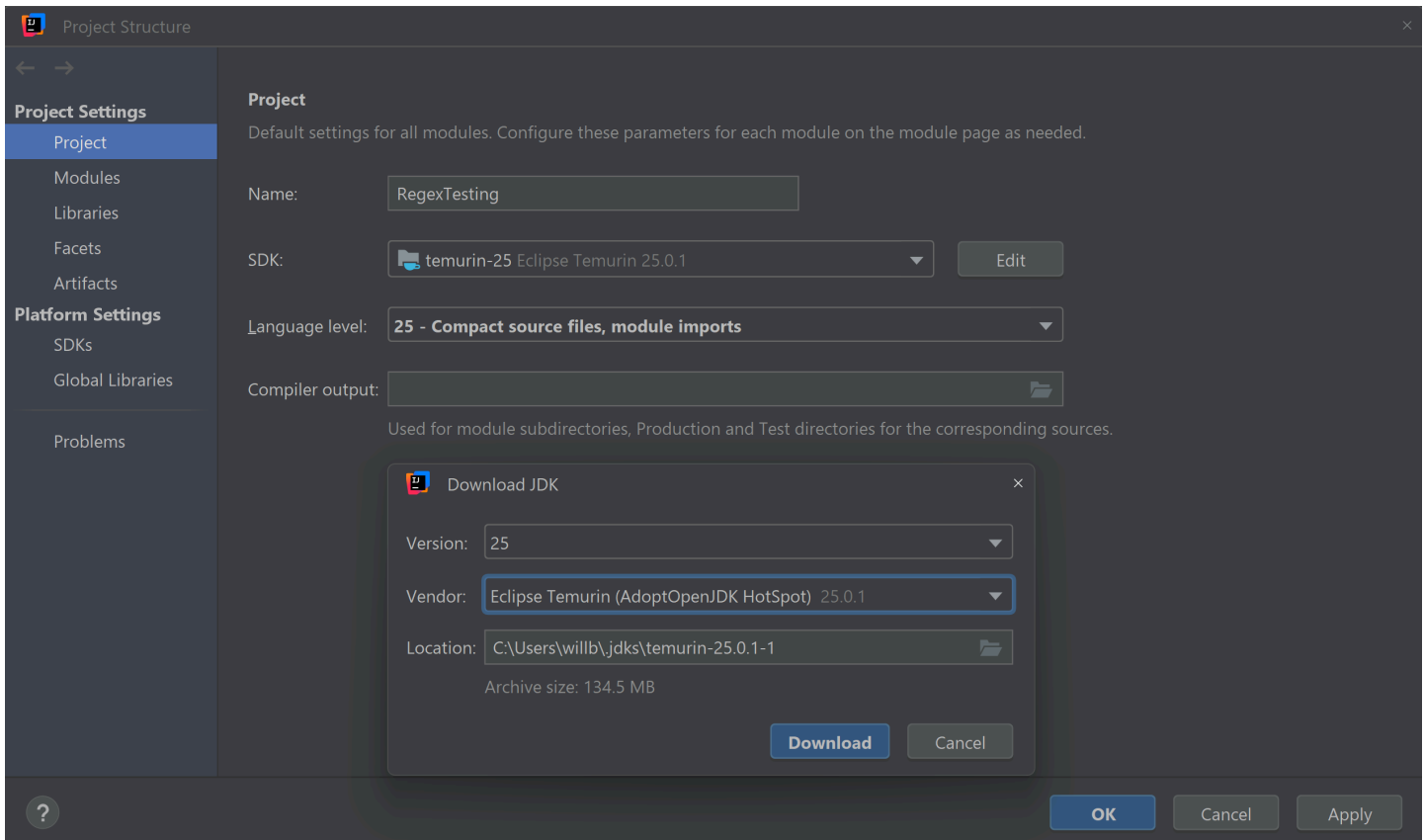


Next, open the project folder in IntelliJ.

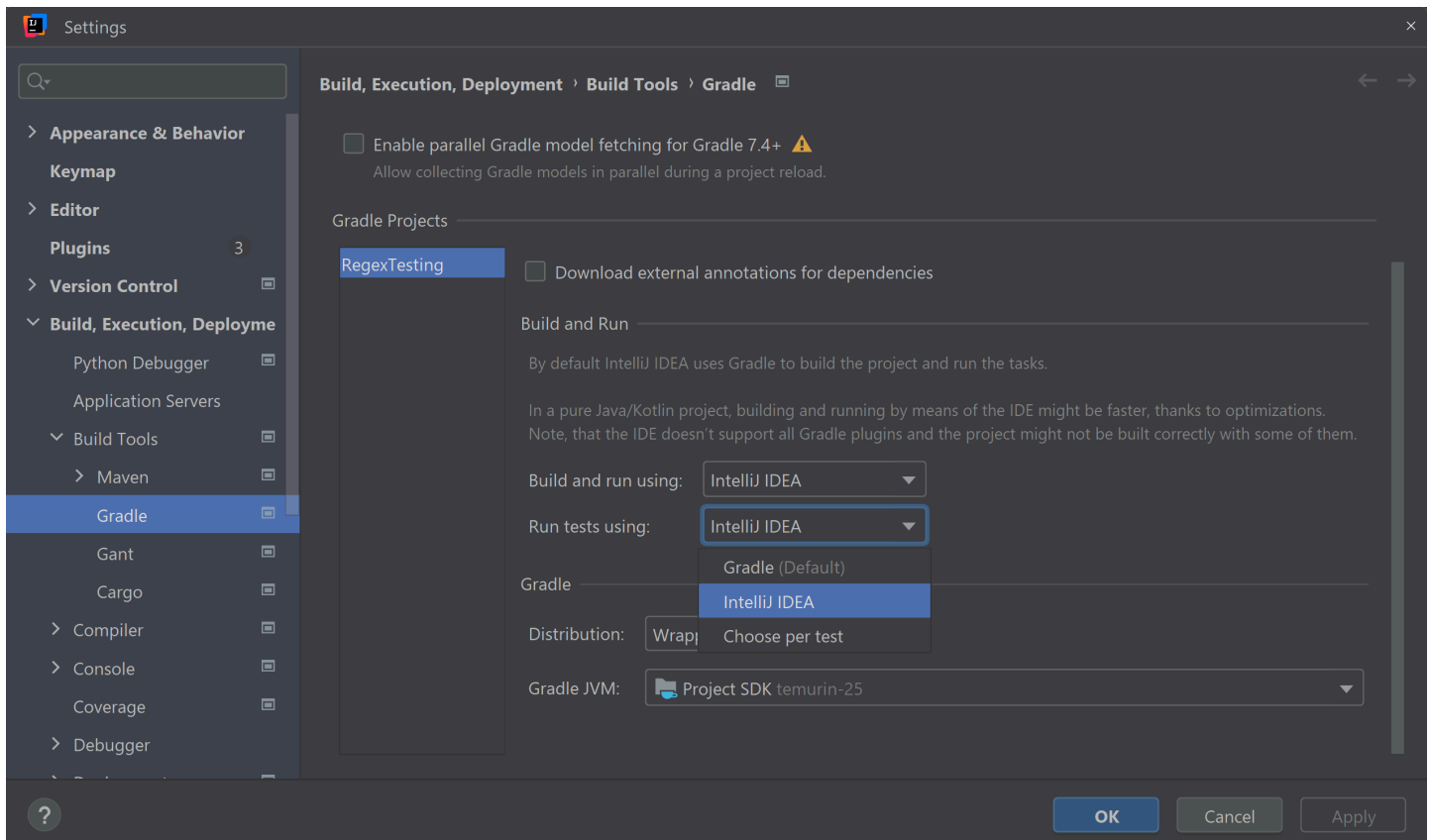


Once opened, navigate to `File -> Project Structure` - this opens up a settings window showing the Java version used for the project. For this course, we will use `Java 25` (SDK) and language level `25` - `Compact source files, module imports` (NOT the `Preview` one; these features may be changed/removed

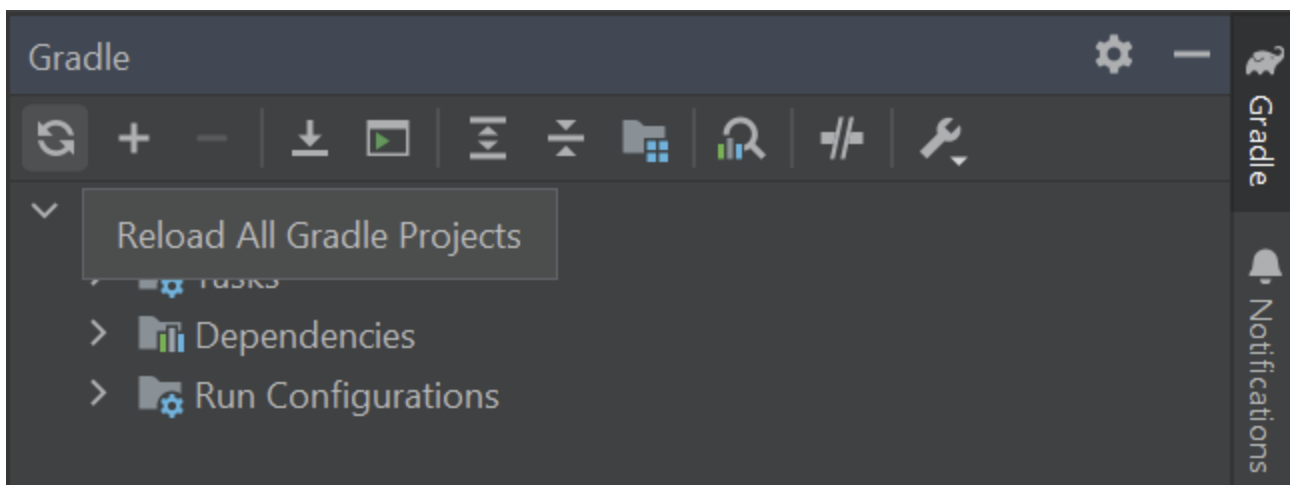
in future versions). The SDK dropdown includes a **Download JDK** option where IntelliJ can download and manage a JDK for you. I personally prefer Eclipse Temurin (previously AdoptOpenJDK) over Oracle's being more open-source, but they should all work the same for our needs.



Lastly, navigate to **File -> Settings** then **Build, Execution, Deployment -> Build Tools -> Gradle** - this contains settings for working with Gradle builds. First, change both **Build and run using** and **Run tests using** to **IntelliJ** instead of the default Gradle - it has a much better user experience, especially for reporting test failures. Finally, verify that **Gradle JVM** is set to the same Java SDK (**25**, which should be the project default) as downloaded earlier - this may be different if you've worked with multiple Java versions like I have.



As a final confirmation, find the **Gradle** tab in the sidebar and **Reload All Gradle Projects**. This ensures everything is still working correctly after configuration changes, especially with changes to the Java version.

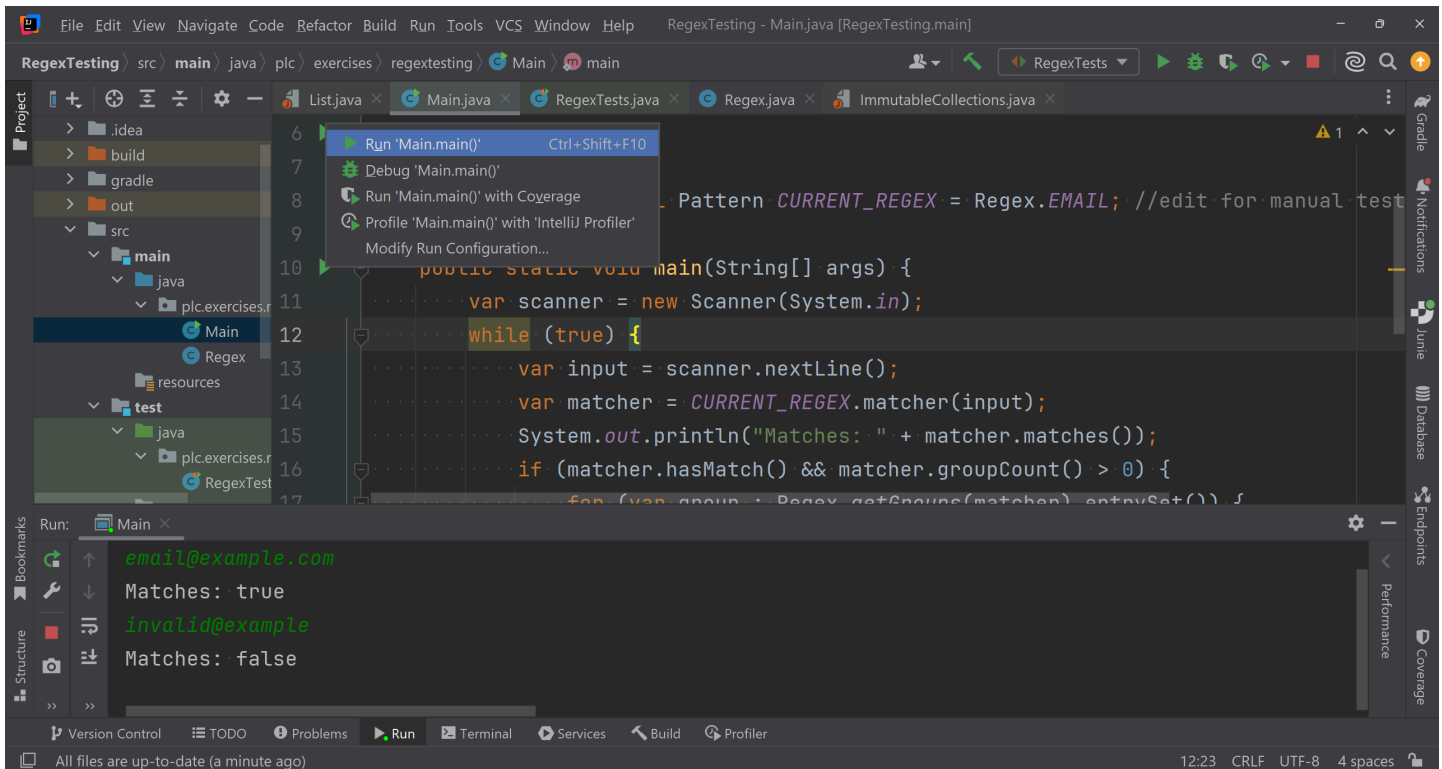


If any failures occur with reloading or trying to run tests below, please ask in office hours. Gradle is great when it works but very painful to debug when issues occur.

Executing Code/Tests

IntelliJ should display a run action next to any potential entry points, such as **main** or JUnit tests. The top-right part of the navigation bar has some additional run utilities for rerunning previous tasks or running with debug/coverage enabled (may be helpful, but these are moderately complex tools).

For `main`, the run window supports input for `System.in` (`stdin`). Assignments will typically provide a `main` method that can be used for manually testing, but is otherwise unused for the rest of the project. `RegexTesting's` `main` will print match information for the input with the current regex.



Tests work similarly, but display an overview of tests results instead (green is passed, yellow is failed, and red is invalid setup). It's generally helpful to run individual test methods to focus on the specific code/tests you're working with (which will be faster too).

